

Product Requirement Document (PRD)

Feature: Recommendation & Personalization System (Phase-1)

Product: myWaves

Prepared by: Product Team – WAVES OTT

Date: 14 Oct 2025

Version: v1.0

1. Overview

myWaves aims to deliver a personalized and engaging video discovery experience to users.

In **Phase-1**, the objective is to introduce a **basic recommendation framework** using three core APIs:

1. **Trending API** – Displays platform-wide trending content.
2. **History Recommendation API** – Displays videos based on user watch history.
3. **Similar-to-This API** – Displays related videos for the currently playing video.

A unified **Homepage Wrapper API** will merge results from Trending and History APIs to generate a personalized homepage feed.

This phase lays the foundation for a future **robust personalization engine (Phase-2 onwards)** using user behavior, demographics, and ML-based models.

2. Objectives

- Introduce a data-driven yet deterministic recommendation layer for the homepage and player pages.

- Deliver a consistent UX across web and mobile.
 - Handle cold-start users effectively using trending data.
 - Ensure modular API design for easy upgrade in future phases.
-

3. Scope (Phase-1)

In Scope

- Trending, History Recommendation, and Similar-to-This APIs.
- Homepage Wrapper API combining trending and history.
- Sorting logic to be defined for other genres/playlists. This to be internally decided by AI/ML team
- API prioritization & merging logic.
- Lazy-loading implementation for homepage and related content sections.
- Integration with desktop and mobile layouts.
- Logging of basic engagement events (impressions, clicks, watch).

Out of Scope (Phase-1)

- Machine learning-based recommendations.
 - Demographic or persona-based personalization.
 - Multi-modal embeddings or advanced feature stores.
 - Real-time adaptive ranking or A/B experimentation (planned for Phase-2).
-

4. User Stories

1. **As a new user**, I should see trending videos on my homepage since I have no watch history.
 2. **As a returning user**, I should see videos based on my watch history prioritized over trending.
 3. **As a viewer**, I should see related videos under the currently playing video that match its genre, tags, or content type.
 4. **As a user on a desktop**, I want to see videos arranged in rows of 3 for better screen utilization.
 5. **As a user on mobile**, I want single-column video rows to improve readability and ease of scrolling.
 6. **As a product analyst**, I want engagement metrics logged (impression, click, watch) to analyze CTR and watch-time uplift.
-

5. Functional Flow

5.1 Homepage Recommendations

- **For new users:**
 - Only **Trending API** results shown.
 - 3 videos per row (desktop), 1 video per row (mobile).
 - Lazy-load enabled beyond the first batch.
- **For returning users:**
 - **Homepage Wrapper API** merges **Trending API** and **History Recommendation API**.
 - History-based recommendations have higher priority.
 - Deduplication between sources.

- Videos arranged per device layout (see UI layout section).

5.2 Related Videos (Player Page)

- **API Used:** Similar-to-This API
- **Display Rules:**
 - **Desktop:** 3 videos per row, total of **12 videos fetched initially**, remaining lazy-loaded.
 - **Mobile:** 1 video per row, total of **4 videos fetched initially**, remaining lazy-loaded.
- **Ordering:** Sorted by `similarity_score`.
- **Sources:** Based on genre, creator affinity, and co-watch patterns.

6. API Architecture (Phase-1)

6.1 Homepage Wrapper API

Endpoint: `/api/homepage?user_id=&device=®ion=&limit_rows=`

Logic:

- Fetch user history count.
- If `history_count < MIN_HISTORY_THRESHOLD`, show only trending.
- If threshold met, fetch both sources → merge with priority for history videos.
- Return structured rows by device type.

Desktop: 3 videos per row → max 6 rows fetched (18 videos).

Mobile: 1 video per row → max 12 rows fetched (12 videos).

Lazy-load additional rows using **next_cursor**.

6.2 Trending API

Endpoint: /api/trending?region=&limit=
Returns trending content for region or global fallback.
Used for homepage and cold-start users.

6.3 History Recommendation API

Endpoint: /api/history_recommendation?user_id=&limit=&cursor=
Returns recommended videos based on:

- Watch history (most watched genres, creators).
 - Likes & completed watch duration.
 - Recency weighting.
-

6.4 Similar-to-This API

Endpoint: /api/similar_to_this?video_id=&limit=
Returns related content for the playing video based on:

- Genre match.
 - Tag overlap.
 - Creator/channel match.
 - Co-watch and embedding similarity.
-

7. Data Model

Video Object

Field	Type	Description
video_id	string	Unique ID
title	string	Video title
thumbnail_url	string	Thumbnail
duration_seconds	integer	Duration
uploader_id	string	Channel ID
genre	string	Video genre
language	string	Video language
published_at	datetime	Publish date
metrics	object	Views, likes, watch time
tags	array	Video tags
source	string	“history” or “trending”
score	float	Computed score

8. Layout & Display Rules

Homepage

Device	Videos per row	Rows fetched initially	Lazy-load trigger
Desktop	3	6 rows (18 videos)	On scroll (end of 5th row)
Mobile	1	12 rows (12 videos)	On scroll (end of 10th video)

Related Content (Player Page)

Device	Videos per row	Videos fetched initially	Lazy-load trigger
--------	----------------	--------------------------	-------------------

Desktop	3	12 videos	On scroll (end of 9th video)
Mobile	1	4 videos	On scroll (end of 3rd video)

9. Prioritization Logic

Condition	API Source	Priority	Notes
New user	Trending only	100%	Until history threshold met
Returning user	History + Trending	History > Trending	Blend weights 70:30
Video player (related)	Similar-to-This	100%	Sorted by similarity score

Thresholds:

- MIN_HISTORY_THRESHOLD = 5 videos watched
 - MIN_WATCH_TIME_THRESHOLD = 10 minutes
 - HISTORY_WEIGHT = 0.7, TRENDING_WEIGHT = 0.3
-

10. Lazy Loading

- Cursor-based pagination supported for all APIs.
 - next_cursor token included in each response.
 - Subsequent calls fetch the next set excluding already returned videos.
 - Backend ensures deduplication per session.
-

11. Event Logging

Events to capture:

- Homepage API call
- Impression event (video visible on screen)
- Click / tap event
- Watch start, watch complete
- Related video click event

Captured data will be used to improve future model training.

12. Non-Functional Requirements

Parameter	Target
Latency	< 500 ms (P95)
Uptime	99.9%
Scalability	1M DAU
API Format	REST / JSON
Security	PII encryption, privacy-compliant
Observability	Request logging, latency metrics, CTR dashboards

13. Future Roadmap (Phase-2 & Beyond)

Phase	Focus	Key Additions
Phase-2	Personalized Recommendations	Collaborative + Content-based hybrid model, feature store, embeddings
Phase-3	Real-time Personalization	Online learning, contextual ranking, A/B testing
Phase-4	Advanced Personalization	Multi-modal embeddings (text, thumbnail), monetization-aware ranking, explainability layer

Future Data Parameters to include:

- Watch-through rate
 - Session context (time/day)
 - Search behavior
 - Explicit preferences (liked genres)
 - Age group, location, gender (optional, with consent)
 - Negative feedback (“Not Interested”)
 - Creator affinity
 - Freshness & diversity score
-

14. Risks & Mitigation

Risk	Impact	Mitigation
Cold-start for new users	Low relevance	Use trending + optional onboarding interests
Filter bubbles	Reduced content diversity	Introduce exploration slots
Latency issues with multiple APIs	Slow homepage load	Cache trending + prefetch history
Privacy concerns	Regulatory risk	Allow opt-out, anonymize user IDs

15. Success Metrics

- CTR increase (%) vs. static homepage
- Average watch time per DAU

- D1 and D7 retention uplift
 - Completion rate (WTR) per session
 - Latency and uptime of API endpoints
-

16. Deliverables

- Trending API
 - History Recommendation API
 - Similar-to-This API
 - Homepage Wrapper API
 - Sorting Logic for other genres
 - Lazy-loading integration
 - Frontend integration (desktop + mobile)
 - Logging & analytics setup
 - Monitoring dashboards
-

Appendix

Figma design - Wavestube

<https://www.figma.com/design/lhMaXvTte8yl7zf5VG9rzY/Waves?node-id=4107-8492&p=f&t=HY1IHvivTApZSzo3-0>

Viewer Journey to be checked

Example Homepage Response

```
{
  "user_id": "U123",
  "rows": [
    {
      "row_id": 1,
      "videos": [
        {"video_id": "v101", "source": "history"},
        {"video_id": "v102", "source": "history"},
        {"video_id": "v103", "source": "trending"}
      ]
    }
  ],
  "next_cursor": "CURSOR_ABC",
  "metadata": {"history_count": 7}
}
```
